DBMS – ADVANCED SQL QUERY IMPLEMENTATIO

PROJECT TITLE: MEESHO – E-COMMERCE ORDER MANAGEMENT SYSTEM

OBJECTIVES

1. Implement subqueries and nested queries.
2. Find products above the average price.
3. Identify customers with maximum purchases.
4. Generate complex business queries.
5. Prepare advanced SQL reports.

1. SUBQUERIES AND NESTED QUERIES

Aim: Display orders whose total amount is greater than the average order amount.

```
SQL> SELECT ORDER_ID, CUSTOMER_ID, TOTAL_AMOUNT, ORDER_STATUS
  2  FROM ORDERS
  3  WHERE TOTAL_AMOUNT > (
  4      SELECT AVG(TOTAL_AMOUNT)
  5      FROM ORDERS
  6  )
  7  ORDER BY TOTAL_AMOUNT DESC;

  ORDER_ID CUSTOMER_ID TOTAL_AMOUNT ORDER_STATUS
---------- ----------- ------------ ------------------------------
       208         108         1599 Delivered
       204         104         1499 Pending
       202         102         1299 Shipped
       206         106         1099 Shipped
```

2. FIND PRODUCTS ABOVE AVERAGE PRICE

Aim: Display products whose price is greater than the average product price.

```
SQL> SELECT PRODUCT_ID, PRODUCT_NAME, PRICE, STOCK_QUANTITY
  2  FROM MEESHO_PRODUCT
  3  WHERE PRICE > (
  4      SELECT AVG(PRICE)
  5      FROM MEESHO_PRODUCT
  6  )
  7  ORDER BY PRICE DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE STOCK_QUANTITY
---------- --------------
       107
Party Wear Gown
       999             10

       108
Designer Kurti
       849             12

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE STOCK_QUANTITY
---------- --------------

       102
Floral Saree
       799             15
```

2.1 FIND PRODUCTS BELOW AVERAGE PRICE

```
SQL> SELECT PRODUCT_ID, PRODUCT_NAME, PRICE, STOCK_QUANTITY
  2  FROM MEESHO_PRODUCT
  3  WHERE PRICE < (
  4       SELECT AVG(PRICE)
  5       FROM MEESHO_PRODUCT
  6  )
  7  ORDER BY PRICE ASC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE STOCK_QUANTITY
---------- --------------
       110
Casual Leggings
       249              0

       103
Leggings
       299             40

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE STOCK_QUANTITY
---------- --------------

       106
Palazzo Pants
       399             30
```

3. IDENTIFY CUSTOMERS WITH MAXIMUM PURCHASES

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         SUM(O.TOTAL_AMOUNT) AS TOTAL_PURCHASE
  4  FROM CUSTOMER_DETAILS C
  5  JOIN ORDERS O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  8  ORDER BY TOTAL_PURCHASE DESC;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_PURCHASE
--------------
        101
Anitha
          1798

        102
Priya
          1299

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_PURCHASE
--------------
```

3.1 FIND THE MAXIMUM PURCHASE

```
SQL> SELECT MAX(TOTAL_PURCHASE) AS MAXIMUM_PURCHASE
  2  FROM (
  3       SELECT CUSTOMER_ID,
  4              SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
  5       FROM ORDERS
  6       GROUP BY CUSTOMER_ID
  7  );

MAXIMUM_PURCHASE
----------------
            1798
```

3.2 DISPLAY CUSTOMER DETAILS WITH MAXIMUM PURCHASE

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         C.EMAIL,
  4         C.PHONE,
  5         SUM(O.TOTAL_AMOUNT) AS TOTAL_PURCHASE
  6  FROM CUSTOMER_DETAILS C
  7  JOIN ORDERS O
  8  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  9  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME,
 10           C.EMAIL, C.PHONE
 11  HAVING SUM(O.TOTAL_AMOUNT) = (
 12      SELECT MAX(TOTAL_PURCHASE)
 13      FROM (
 14          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
 15          FROM ORDERS
 16          GROUP BY CUSTOMER_ID
 17      )
 18  );

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
PHONE      TOTAL_PURCHASE
---------- --------------
        101
Anitha

9876543210           1798
```

4. CUSTOMERS WHO PLACED ORDERS ABOVE AVERAGE ORDER VALUE

```
SQL> SELECT DISTINCT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         C.EMAIL,
  4         C.PHONE
  5  FROM CUSTOMER_DETAILS C
  6  JOIN ORDERS O
  7  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  8  WHERE O.TOTAL_AMOUNT > (
  9      SELECT AVG(TOTAL_AMOUNT)
 10      FROM ORDERS
 11  )
 12  ORDER BY C.CUSTOMER_ID;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
PHONE
----------
        102
Priya

9876543211
```

5. FIND PRODUCTS WHOSE SALES ARE ABOVE AVERAGE

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         SUM(OD.QUANTITY) AS TOTAL_QUANTITY,
  4         SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_SALES
  5  FROM MEESHO_PRODUCT P
  6  JOIN ORDER_DETAILS OD
  7  ON P.PRODUCT_ID = OD.PRODUCT_ID
  8  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  9  HAVING SUM(OD.QUANTITY * OD.PRICE) > (
 10      SELECT AVG(PRODUCT_SALES)
 11      FROM (
 12          SELECT SUM(QUANTITY * PRICE) AS PRODUCT_SALES
 13          FROM ORDER_DETAILS
 14          GROUP BY PRODUCT_ID
 15      )
 16  )
 17  ORDER BY TOTAL_SALES DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY TOTAL_SALES
-------------- -----------
       101
Cotton Kurti
             3        1500

       102
Floral Saree
             5        1000
```

6. CUSTOMER PURCHASE REPORT

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS,
  4         SUM(O.TOTAL_AMOUNT) AS TOTAL_PURCHASE,
  5         AVG(O.TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE
  6  FROM CUSTOMER_DETAILS C
  7  JOIN ORDERS O
  8  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  9  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
 10  ORDER BY TOTAL_PURCHASE DESC;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS TOTAL_PURCHASE AVERAGE_ORDER_VALUE
---------------- -------------- -------------------
         101
Anitha
                2           1798                 899

         102
Priya
                1           1299                1299

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS TOTAL_PURCHASE AVERAGE_ORDER_VALUE
---------------- -------------- -------------------
```

7.ADVANCED PRODUCT SALES REPORT

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         COUNT(DISTINCT OD.ORDER_ID) AS NUMBER_OF_ORDERS,
  4         SUM(OD.QUANTITY) AS TOTAL_QUANTITY,
  5         SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_SALES,
  6         AVG(OD.PRICE) AS AVERAGE_SELLING_PRICE
  7  FROM MEESHO_PRODUCT P
  8  JOIN ORDER_DETAILS OD
  9  ON P.PRODUCT_ID = OD.PRODUCT_ID
 10  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
 11  ORDER BY TOTAL_SALES DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS TOTAL_QUANTITY TOTAL_SALES AVERAGE_SELLING_PRICE
---------------- -------------- ----------- --------------------
       101
Cotton Kurti
               2              3        1500                  500

       102
Floral Saree
               1              5        1000                  200

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS TOTAL_QUANTITY TOTAL_SALES AVERAGE_SELLING_PRICE
---------------- -------------- ----------- --------------------
```

8. CATEGORY-WISE ADVANCED SALES REPORT

```
SQL> SELECT C.CATEGORY_ID,
  2         C.CATEGORY_NAME,
  3         COUNT(DISTINCT P.PRODUCT_ID) AS NUMBER_OF_PRODUCTS,
  4         SUM(OD.QUANTITY) AS TOTAL_QUANTITY,
  5         SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_SALES,
  6         AVG(OD.PRICE) AS AVERAGE_SELLING_PRICE
  7  FROM CATEGORY C
  8  JOIN MEESHO_PRODUCT P
  9  ON C.CATEGORY_ID = P.CATEGORY_ID
 10  JOIN ORDER_DETAILS OD
 11  ON P.PRODUCT_ID = OD.PRODUCT_ID
 12  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
 13  ORDER BY TOTAL_SALES DESC;

CATEGORY_ID CATEGORY_NAME
----------- --------------------------------------------------
NUMBER_OF_PRODUCTS TOTAL_QUANTITY TOTAL_SALES AVERAGE_SELLING_PRICE
------------------ -------------- ----------- --------------------
          1 Electronics
                 2              5        1700           366.666667
```

CONCLUSION

Successfully implemented advanced SQL queries in the Meesho E-Commerce Order Management System using subqueries, nested queries, aggregate functions, joins, and grouping operations. Generated customer purchase, product sales, and category-wise sales reports.